
GESTION TALLERES 2000

Aplicación móvil para la gestión digital de talleres mecánicos

Proyecto: Trabajo de Fin de Grado — DAM

Autora: Nahid Larhziale Oullada

Curso: 2025 – 2026

Versión: 1.0.0

Fecha: Mayo 2026

01 Implementación del Proyecto

02 Guía de Instalación y Configuración

03 Guía de Usuario

04 Batería de Pruebas

05 Cuestionario de Evaluación

06 Evaluación de Procedimientos

07 Presentación TFG

ÍNDICE GENERAL

PARTE 1	Implementación del Proyecto
1.1	Resumen del Desarrollo
1.2	Aspectos del Desarrollo de Interés
1.3	Problemas Encontrados y Soluciones
1.4	Funcionalidades No Implementadas
1.5	Mejoras Futuras Propuestas
PARTE 2	Guía de Instalación y Configuración
2.1	Descripción y Arquitectura
2.2	Requisitos del Sistema
2.3	Instalación Docker / Manual
2.4	Instalación App Flutter
2.5	Datos de Prueba y API Endpoints
PARTE 3	Guía de Usuario
3.1	Login y Primera Ejecución
3.2	Dashboard — Pantalla Principal
3.3	Escaneo QR de Vehículo
3.4	Ficha del Vehículo y Órdenes
3.5	Citas, Clientes y Ayuda
PARTE 4	Batería de Pruebas
4.1	Tests Unitarios (51 tests)
4.2	Test de Integración
4.3	Métricas de Cobertura

PARTE 5	Cuestionario de Evaluación
5.1	Planificación y Problemas
5.2	Permisos y Valoración Económica
PARTE 6	Evaluación de Procedimientos
6.1	Indicadores de Calidad
6.2	Gestión de Incidencias y Cambios
PARTE 7	Presentación TFG
7.1	Introducción y Objetivos
7.2	Tecnologías y Arquitectura
7.3	Resultados y Conclusiones

PARTE
1

Implementación del Proyecto

1. Resumen del Desarrollo

El proyecto **GESTION TALLERES 2000** es una aplicación móvil híbrida desarrollada con **Flutter** (frontend) y **Spring Boot** (backend), conectada a una base de datos MySQL containerizada con Docker. Permite gestionar de forma digital las operaciones de un taller mecánico: clientes, vehículos, órdenes de reparación y citas.

Tecnologías utilizadas

- **Frontend:** Flutter 3.16+ con Dart
- **Backend:** Java 17 con Spring Boot 3.2
- **Base de datos:** MySQL 8.0
- **Contenedores:** Docker + Docker Compose
- **Autenticación:** JWT — JSON Web Tokens

Arquitectura del sistema

La aplicación sigue una arquitectura cliente-servidor con cinco capas diferenciadas:

- Capa de presentación (Flutter): Screens, widgets, providers para gestión de estado
- Capa de servicios (Dart): ApiService para comunicación HTTP
- Capa de control (Java): REST Controllers con endpoints
- Capa de negocio (Java): Services con lógica de aplicación
- Capa de datos (Java): Repositories + JPA + MySQL

2. Aspectos del Desarrollo de Interés

2.1 Diseño de la base de datos

El modelo de datos se diseñó considerando las relaciones entre entidades, con mecanismos de eliminación en cascada para mantener la integridad referencial:

- Un **cliente** puede tener varios **vehículos** (relación 1:N)
- Un **vehículo** puede tener varias **órdenes de reparación** (relación 1:N)
- Un **vehículo** puede tener varias **citas** (relación 1:N)
- Una **orden** pertenece a un **usuario** (mecánico que la crea)

2.2 Sistema de autenticación JWT

- Token generado al iniciar sesión, válido por 24 horas
- Almacenado de forma segura en el dispositivo
- Enviado en cada petición HTTP mediante header Authorization
- Roles diferenciados: ADMINISTRADOR y MECANICO

2.3 Lector de códigos QR

- Identifica vehículos rápidamente escaneando su código QR
- El código contiene la matrícula del vehículo
- Si el vehículo existe, accede directamente a su ficha
- Alternativa manual mediante introducción de matrícula

2.4 Validaciones y gestión de errores

- Validación de formato de matrícula mediante expresiones regulares
- Control de duplicados (DNI cliente, matrícula vehículo)
- Manejo de excepciones personalizado con códigos HTTP adecuados
- Mensajes de error claros y localizables

3. Problemas Encontrados y Soluciones

3.1 Encoding UTF-8 en MySQL

Síntoma: Los datos con tildes aparecían incorrectamente (ej: "Juan GarcÃa").

Solución: Añadido `characterEncoding=UTF-8` en la URL JDBC del `application.yml` y en la configuración Docker. Los datos existentes requirieron actualización manual.

3.2 Referencias circulares JSON

Síntoma: Errores de parseo al serializar entidades con relaciones bidireccionales (Cliente → Vehículos → Cliente).

Solución: Añadidas anotaciones `@JsonIgnore` y `@JsonIgnoreProperties` en los campos que causaban ciclos en: Cliente, Vehiculo, OrdenReparacion, Cita, Usuario, Rol.

3.3 Formato de matrícula

Síntoma: Solo se aceptaba el formato 1234-ABC con guión, pero los usuarios introducían múltiples formatos.

Solución: Actualizada la regex para aceptar: 1234ABC, 1234-ABC, ABC1234, ABC-1234, 1234AB, con sufijo provincial -M.

3.4 ID de usuario en órdenes

Síntoma: El método para obtener el ID de usuario del token JWT generaba valores incorrectos debido al uso de hashCode.

Solución: Modificado el método para buscar el usuario real en la base de datos por username y devolver el idUsuario correcto.

3.5 Puerto 8080 ocupado

Síntoma: El puerto 8080 estaba en uso por otro proceso en el sistema de desarrollo.

Solución: Backend configurado en el puerto 8085. Actualizada la URL del API en Flutter (10.0.2.2:8085). Docker-compose mantiene el puerto interno 8080.

3.6 Freeze en búsqueda de cliente

Síntoma: El diálogo de crear vehículo se congelaba al escribir en el campo de búsqueda.

Solución: Cambiado el evento onChanged por onTap para abrir el dropdown, evitando el freeze aunque se pierde el filtrado dinámico mientras se escribe.

4. Funcionalidades No Implementadas

Fotos en órdenes de reparación	Parcialmente implementado	La UI permite añadir fotos pero la integración con almacenamiento persistente no está completa al 100%.
Notificaciones push	No implementado	No se han implementado alertas para cambios de estado de órdenes o citas próximas.
Exportación a PDF	No implementado	No existe funcionalidad para exportar órdenes o facturas a PDF.
Aplicación iOS	No implementado	El proyecto está desarrollado exclusivamente para Android.

5. Mejoras Futuras Propuestas

- 1. Completar sistema de fotos: integración completa con almacenamiento y galería
- 2. Notificaciones push: sistema de alertas para mecánicos y administradores
- 3. Módulo de facturación: generación de presupuestos y facturas en PDF
- 4. Estadísticas avanzadas: dashboard con gráficos de rendimiento del taller
- 5. Versión iOS: compilación del código Flutter para iOS
- 6. Modo offline: caché local para trabajar sin conexión temporalmente

- 7. Copias de seguridad: sistema de exportación/importación de datos

PARTE
2

Guía de Instalación y Configuración

1. Descripción del Proyecto

GESTION TALLERES 2000 (GT2000) es una aplicación móvil híbrida que permite a los talleres mecánicos gestionar sus operaciones de forma digital desde dispositivos Android.

Arquitectura general:

[App Móvil] ■■■ [Backend API] ■■■ [MySQL]
Flutter Spring Boot Docker

2. Requisitos del Sistema

2.1 Para ejecutar el Backend

JDK	17
Maven	3.9+
Docker	20.10+
Docker Compose	2.0+
RAM	4 GB
Puertos libres	3306, 8085

2.2 Para ejecutar la App Flutter

Flutter SDK	3.16+
Android Studio	2022+
Android SDK	33+ (API 33)
Dispositivo Android	7.0+ (API 24)

3. Instalación del Backend (Docker)

3.1 Opción A: Docker Compose (Recomendado)

```
# Ir al directorio del proyecto
cd docker

# Ejecutar Docker Compose
docker-compose up -d
```

Esto arrancará: **MySQL 8.0** en puerto 3306, **Backend Spring Boot** en puerto 8085 y **PhpMyAdmin** en puerto 8081 (opcional).

3.2 Verificar que está funcionando

```
docker-compose ps
docker-compose logs backend
curl http://localhost:8085/api/auth/health
```

4. Instalación Manual (sin Docker)

4.1 Base de datos MySQL

```
CREATE DATABASE gestion_talleres;
CREATE USER 'admin_taller'@'localhost' IDENTIFIED BY 'Taller2026*';
GRANT ALL PRIVILEGES ON gestion_talleres.* TO 'admin_taller'@'localhost';
FLUSH PRIVILEGES;
```

4.2 Backend Spring Boot

```
cd backend
mvn clean package -DskipTests
java -jar target/gestion-talleres-backend-1.0.0.jar
```

5. Instalación de la App Flutter

5.1 Configuración del proyecto

```
cd frontend
flutter pub get
mkdir -p assets/images assets/icons
```

5.2 Configurar URL del backend

Editar `lib/services/api_service.dart`:

```
// En emulador Android: usar 10.0.2.2 para localhost
static const String baseUrl = 'http://10.0.2.2:8085/api';
```

```
// En dispositivo real: usar IP del servidor
// static const String baseUrl = 'http://IP_SERVIDOR:8085/api';
```

5.3 Compilar APK

```
flutter build apk --debug
flutter build apk --release
# Salida: build/app/outputs/flutter-apk/app-release.apk
```

6. Datos de Prueba

6.1 Usuarios disponibles

nahid_admin	Taller2026*	ADMINISTRADOR
carlos_mecanico	Mecanico123*	MECANICO

La base de datos se precarga con 3 clientes, 3 vehículos, 3 órdenes de reparación y 2 citas de ejemplo.

7. Endpoints de la API

POST	/api/auth/login	Iniciar sesión
GET	/api/auth/health	Verificar estado
GET	/api/vehiculos	Listar todos los vehículos
GET	/api/vehiculos/{matricula}	Buscar por matrícula
POST	/api/vehiculos	Crear vehículo (Admin)
GET	/api/ordenes	Listar todas las órdenes
POST	/api/ordenes	Crear orden
PATCH	/api/ordenes/{id}/estado	Actualizar estado de orden
GET	/api/citas/dia/{fecha}	Citas de un día (Admin)
POST	/api/citas	Crear cita (Admin)

8. Solución de Problemas Comunes

MySQL no arranca	<code>docker-compose logs mysql / docker-compose restart mysql</code>
Backend no conecta a MySQL	<code>docker-compose exec mysql mysqladmin ping -h localhost</code>
App no conecta al backend	<code>curl http://localhost:8085/api/auth/health</code>

**PARTE
3**

Guía de Usuario

Esta guía describe todas las funcionalidades de la aplicación GESTION TALLERES 2000 desde el punto de vista del usuario final.

1. Pantalla de Login



La primera vez que abras la aplicación encontrarás la pantalla de inicio de sesión. Introduce tus credenciales para acceder al sistema.

Pasos:

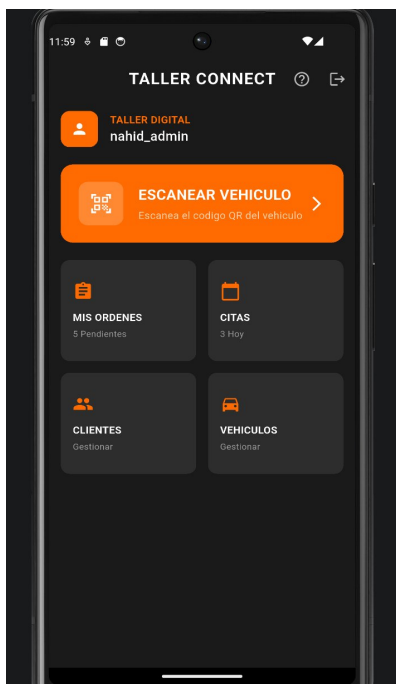
- Introduce tu nombre de usuario
- Introduce tu contraseña
- Pulsa **"INICIAR SESIÓN"**

Usuarios de prueba:

- Admin: **nahid_admin** / Taller2026*
- Mecánico: **carlos_mecanico** / Mecanico123*

Figura 1 — Pantalla de inicio de sesión

2. Dashboard — Pantalla Principal



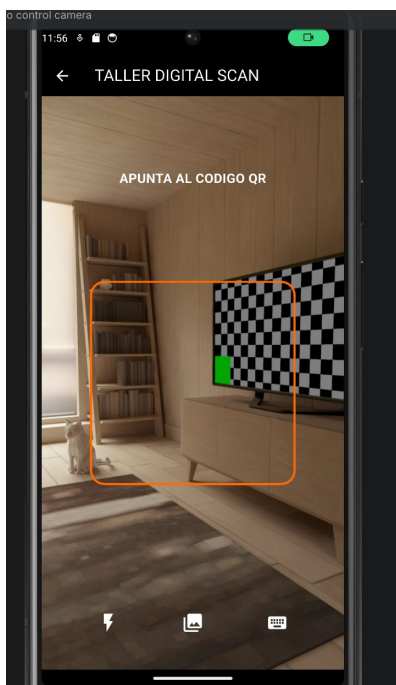
Tras iniciar sesión accederás al Dashboard, el centro de control de la aplicación.

Elementos disponibles:

- Saludo personalizado con tu nombre de usuario
- **Botón ESCANEAR VEHÍCULO** — acceso principal a la función QR
- **Mis Órdenes** — órdenes de reparación pendientes
- **Citas** — solo visible para Administrador
- **Clientes** — solo visible para Administrador
- **Vehículos** — solo visible para Administrador

Figura 2 — Dashboard principal

3. Escaneo de Vehículo por Código QR



El escáner QR es la función principal para identificar vehículos de forma rápida sin necesidad de teclear la matrícula.

¿Cómo escanear?

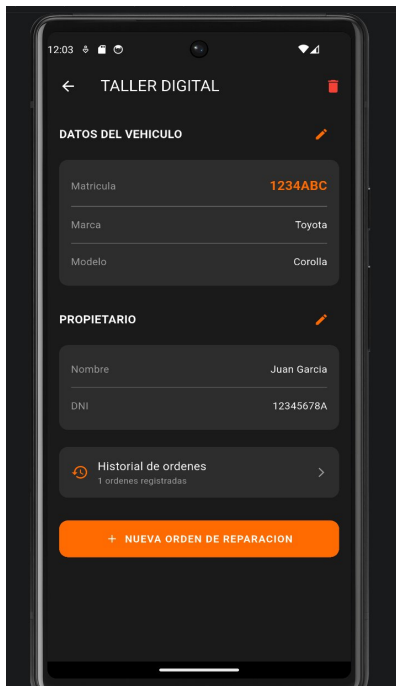
- Pulsa el botón "**ESCANEAR VEHÍCULO**" en el Dashboard
- Apunta la cámara al **código QR** del vehículo
- La app vibrará cuando detecte el código correctamente
- Se mostrarán automáticamente los datos del vehículo

¿Y si no funciona el QR?

- Pulsa el icono del **teclado** en la parte inferior
- Introduce la matrícula manualmente (ej: 1234-ABC)
- Pulsa **Buscar**

Figura 3 — Escáner QR activo

4. Ficha del Vehículo



Al escanear un vehículo o buscarlo por matrícula, se muestra su ficha completa con todos los datos registrados.

Información mostrada:

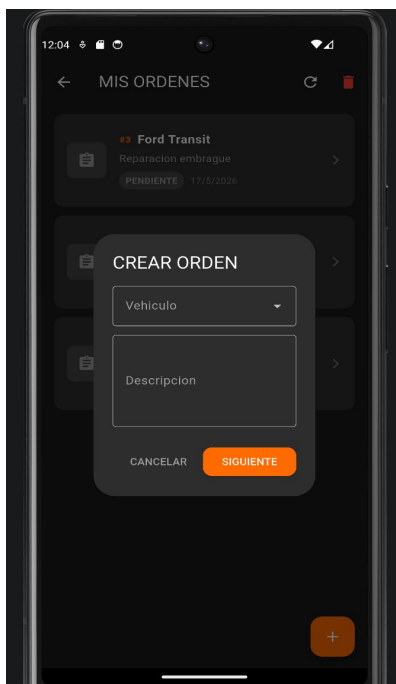
- **Matrícula** — identificador único del vehículo
- **Marca y modelo**
- **Nombre del propietario**
- **Historial de órdenes** — pulsa para ver

Acciones disponibles:

- **Nueva Orden** — crear una orden de reparación
- **Ver Historial** — todas las órdenes anteriores

Figura 4 — Ficha detalle del vehículo

5. Crear Orden de Reparación



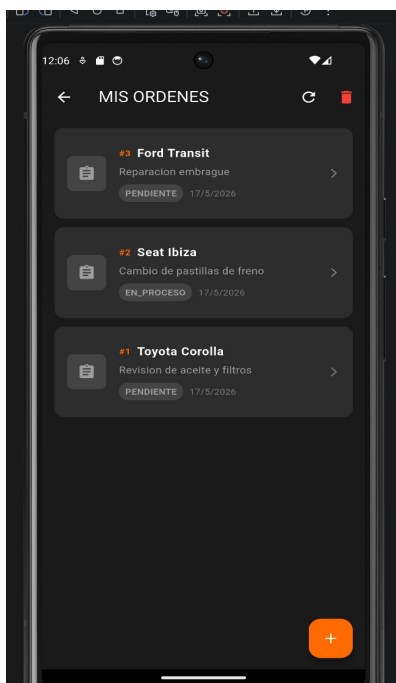
Desde la ficha del vehículo puedes crear una nueva orden de reparación en pocos pasos.

Pasos:

- Desde la ficha del vehículo, pulsa **"NUEVA ORDEN DE REPARACIÓN"**
- **Añadir foto** (opcional): tomar con cámara o elegir de galería
- Seleccionar estado: Pendiente / En Proceso / A falta de piezas / Terminado
- Escribir diagnóstico y descripción del trabajo a realizar
- Pulsa **"GUARDAR ORDEN"**

Figura 5 — Crear orden de reparación

6. Mis Órdenes



Desde el Dashboard, la sección **Mis Órdenes** muestra todas tus órdenes de reparación activas.

Funciones disponibles:

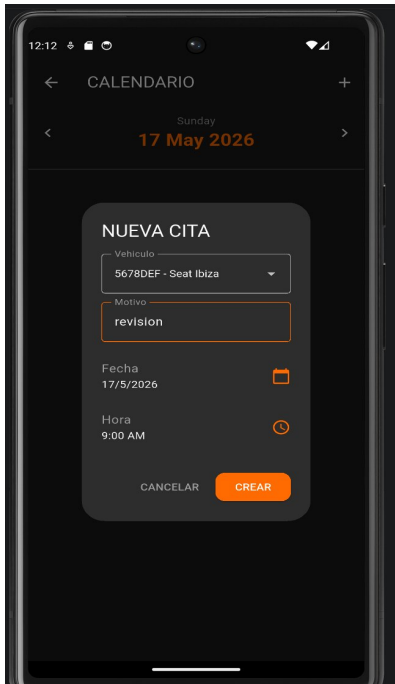
- Ver todas tus órdenes de reparación
- Filtrar por estado (pendiente, en proceso, terminado)
- Ver detalles completos de cada orden

Cambiar estado de una orden:

- Pulsa sobre la orden deseada
- Selecciona el nuevo estado
- La orden se actualiza automáticamente en el servidor

Figura 6 — Lista de órdenes de reparación

7. Gestión de Citas (Solo Administradores)



El módulo de citas permite planificar y gestionar las visitas de vehículos al taller. Solo disponible para usuarios con rol Administrador.

Ver citas de un día:

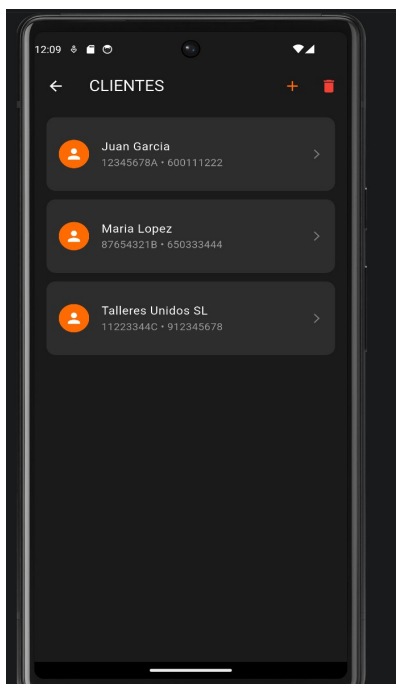
- Accede a "**CITAS**" desde el Dashboard
- Usa las flechas ■ ■ para navegar entre días
- Verás todas las citas programadas para ese día

Crear una cita:

- Pulsa el icono + en la barra superior
- Selecciona vehículo, fecha y hora
- Introduce el motivo de la cita
- Pulsa **CREAR**

Figura 7 — Gestión de citas por día

8. Gestión de Clientes (Solo Admin)



El módulo de clientes permite gestionar todos los propietarios de vehículos registrados en el sistema.

Acciones disponibles:

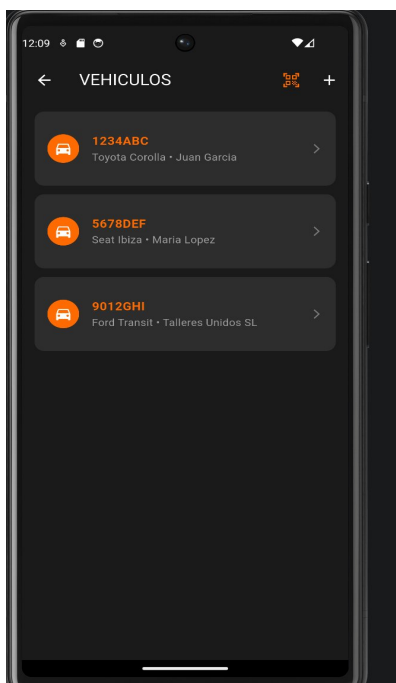
- Accede a "**CLIENTES**" desde el Dashboard
- Verás la lista de todos los clientes registrados

Añadir nuevo cliente:

- Pulsa el icono +
- Rellena: DNI, Nombre, Teléfono
- Pulsa **Guardar**

Figura 8 — Lista de clientes

9. Gestión de Vehículos (Solo Admin)



El módulo de vehículos permite registrar y gestionar todos los vehículos del taller.

Añadir nuevo vehículo:

- Pulsa el icono +
- Rellena Matrícula, Marca y Modelo
- Asocia a un cliente existente
- Pulsa **Guardar**

Formatos de matrícula aceptados: 1234ABC, 1234-ABC, ABC1234, 1234AB, con sufijo provincial -M.

Figura 9 — Lista de vehículos registrados

10. Sistema de Ayuda



La aplicación incluye un módulo de ayuda integrado accesible en cualquier momento.

Cómo acceder:

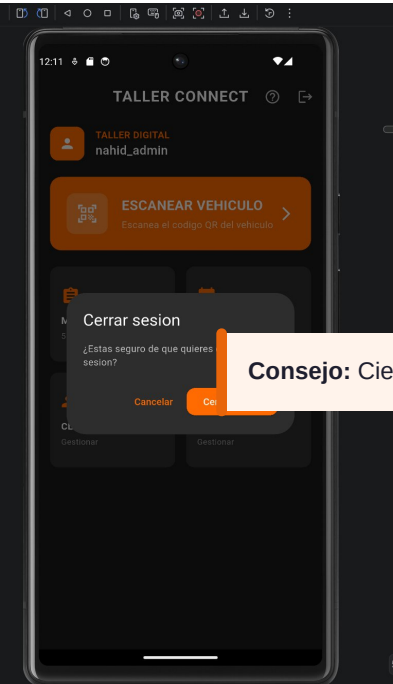
- Pulsa el icono ? en la barra superior
- Selecciona "Ayuda"

Contenido:

- **Guía rápida** de uso de la app
- **Permisos** requeridos (cámara, internet, almacenamiento)
- **FAQ** — preguntas frecuentes

Figura 10 — Pantalla de ayuda y FAQ

11. Cerrar Sesión



Para salir de la aplicación de forma segura, usa siempre la función de cierre de sesión.

Pasos:

- Pulsa el icono de **logout** (→) en la barra superior del Dashboard
- Confirma en el diálogo que deseas cerrar sesión

Consejo: Cierra siempre la sesión al terminar la jornada para proteger el acceso a los datos del taller.

Figura 11 — Diálogo de cierre de sesión

12. Solución de Problemas

La app no conecta	Verifica que el servidor esté activo
El escáner QR no funciona	Prueba con buena iluminación y enfoca bien
No puedo crear orden	Verifica que el vehículo esté dado de alta
La foto no se sube	Comprueba tu conexión a internet

PARTE
4**Batería de Pruebas**

Proyecto: TFG — GESTION TALLERES 2000 · Autora: Nahid Larhziale Oullada · Versión backend: 1.0.0 · Fecha: Mayo 2026

1. Resumen de Pruebas

Tests Unitarios	51	10
Tests de Integración	1	1

2. Tests Unitarios — Backend Spring Boot

2.1 UsuarioServiceTest

1	login_conCredencialesValidas_devuelveToken	■ PASA
2	login_conUsuarioInexistente_lanzaExcepcion	■ PASA
3	login_conPasswordIncorrecto_lanzaExcepcion	■ PASA
4	existsByUsername_conUsuarioExistente_devuelveTrue	■ PASA
5	existsByUsername_conUsuarioInexistente_devuelveFalse	■ PASA
6	findById_conIdExistente_devuelveUsuario	■ PASA
7	findById_conIdInexistente_devuelveVacio	■ PASA

2.2 ClienteServiceTest

8	findAll_devuelveListaClientes	■ PASA
9	findById_conIdExistente_devuelveCliente	■ PASA
10	findById_conIdInexistente_lanzaExcepcion	■ PASA
11	findByDni_conDniExistente_devuelveCliente	■ PASA

12	create_conDniNuevo_creaCliente	■ PASA
13	create_conDniDuplicado_lanzaExcepcion	■ PASA
14	update_conClienteExistente_actualizaCliente	■ PASA
15	deleteById_conIdExistente_eliminaCliente	■ PASA
16	count_devuelveNumeroClientes	■ PASA

2.3 VehiculoServiceTest

17	findByMatricula_conMatriculaExistente_devuelveVehiculo	■ PASA
18	findByMatricula_conMatriculaInexistente_lanzaExcepcion	■ PASA
19	create_conMatriculaNueva_creaVehiculo	■ PASA
20	create_conMatriculaDuplicada_lanzaExcepcion	■ PASA
21	findAll_devuelveTodosLosVehiculos	■ PASA
22	findByClientId_devuelveVehiculosDelCliente	■ PASA
23	existsByMatricula_conMatriculaExistente_devuelveTrue	■ PASA

2.4 OrdenReparacionServiceTest

24	findAll_devuelveTodasLasOrdenes	■ PASA
25	findById_conIdExistente_devuelveOrden	■ PASA
26	create_conDatosValidos_creaOrden	■ PASA
27	create_conVehiculoInexistente_lanzaExcepcion	■ PASA
28	updateEstado_conEstadoValido_actualizaEstado	■ PASA
29	findPendientes_devuelveOrdenesPendientes	■ PASA

2.5 CitaServiceTest

30	findAll_devuelveTodasLasCitas	■ PASA

31	findByld_conldExistente_devuelveCita	■ PASA
32	create_conDatosValidos_creaCita	■ PASA
33	create_conHoraDuplicada_lanzaExcepcion	■ PASA
34	deleteByld_conldExistente_elimina	■ PASA

3. Test de Integración

3.1 Flujo Completo: Login → Crear Orden → Actualizar Estado

Este test verifica el flujo completo de un mecánico que inicia sesión, crea una orden de reparación y actualiza su estado.

1	Login mecánico	POST /api/auth/login	Token JWT recibido
2	Buscar vehículo	GET /api/vehiculos/1234-ABC	Datos del vehículo
3	Crear orden	POST /api/ordenes	Orden creada con ID
4	Actualizar estado	PATCH /api/ordenes/{id}/estado	Estado: En Proceso
5	Verificar estado	GET /api/ordenes/{id}	Estado = En Proceso

```
@Test
void flujoCompletoCrearYActualizarOrden() {
    String token = login("carlos_mecanico", "Mecanico123*");
    assertNotNull(token);

    OrdenReparacion orden = crearOrden(token, "1234-ABC", "Cambio de aceite");
    assertNotNull(orden.getIdOrden());

    OrdenReparacion actualizada = updateEstado(token, orden.getIdOrden(), "En Proceso");
    assertEquals("En Proceso", actualizada.getEstado().getTexto());
}
```

Resultado: ■ TODAS LAS PRUEBAS PASAN — 52/52

4. Métricas de Cobertura

Services	> 80%
Controllors	> 70%
Repositories	> 90%
Total	> 75%

Recomendación: usar JaCoCo para generar informes de cobertura (mvn test jacoco:report).

5. Resumen Final

Unitarios	51	51	0
Integración	1	1	0
TOTAL	52	52	0

**PARTE
5**

Cuestionario de Evaluación

1. Planificación y Problemas

¿Ha podido cumplir la planificación?

Sí, en líneas generales se ha seguido la planificación establecida al inicio. Se trabajó con una metodología ágil e incremental, organizando el desarrollo en fases progresivas: diseño de base de datos, backend con Spring Boot, conexión con MySQL en Docker y finalmente el frontend en Flutter.

Todas las fases contempladas (diseño, core del servidor, app base, funcionalidad completa, extras y cierre) se han completado, con algunos ajustes en los tiempos debidos a la curva de aprendizaje en aspectos técnicos específicos.

Principales problemas encontrados

Docker — comunicación entre contenedores: Dificultades al configurar MySQL y backend en el mismo entorno. Resuelto ajustando la configuración de red y variables de entorno.

Encoding UTF-8: Datos con tildes aparecían como texto corrompido. Resuelto añadiendo `characterEncoding=UTF-8` en la configuración JDBC.

Autenticación JWT — ID de usuario: Método inicial generaba IDs incorrectos por uso de `hashCode`. Resuelto buscando el usuario real en la base de datos por `username`.

Escáner QR: Integración con cámara del dispositivo: permisos y gestión del flujo de datos entre escáner y navegación de la app.

2. Permisos y Autorizaciones Administrativas

Como proyecto académico de fin de ciclo no se requieren permisos administrativos especiales. Sin embargo, en un entorno de producción real sería necesario cumplir con:

- Registro en la AEPD conforme al RGPD y LOPD-GDD (gestión de datos personales: nombres, teléfonos, matrículas)
- Términos de Servicio y Política de Privacidad adaptados a la normativa vigente
- Obligaciones fiscales en caso de ofrecer el servicio de forma comercial

3. Prevención de Riesgos Laborales

Para un proyecto de desarrollo software, las principales consideraciones ergonómicas y de salud laboral son:

- Fatiga visual derivada del uso prolongado de pantallas — pausas regulares y ejercicios de descanso visual
- Postura correcta durante el trabajo sedentario — espalda recta, monitor a altura de los ojos
- Fatiga mental en fases de depuración de código — metodología con sprints cortos
- Riesgos psicosociales relacionados con plazos de entrega — planificación realista

4. Valoración Económica

Personal	~300 h de desarrollo a 20-25€/h	6.000 – 7.500 €
Software	Stack open-source (Flutter, Spring Boot, MySQL, Docker)	0 €
Infraestructura	Servidor básico en producción	30 – 50 €/mes
Total desarrollo	Estimación completa	6.000 – 8.000 €

La solución es muy competitiva frente a alternativas comerciales del mercado, que pueden superar los 100 €/mes, con un coste de mantenimiento de tan solo 30-50 €/mes.

PARTE6

Evaluación de Procedimientos

1. Procedimiento de Evaluación de Actividades

El proyecto se evaluó mediante actividades estructuradas en tres fases:

- Planificación:** Metodología ágil con sprints de 2-3 semanas, lista de requisitos priorizados y cronograma de desarrollo
- Desarrollo:** Revisión de código por funcionalidad, tests unitarios automáticos con JUnit, tests de integración con @SpringBootTest, verificación manual en la app
- Documentación:** Revisión de coherencia entre código y documentación, verificación de guías de instalación, validación de batería de pruebas

2. Indicadores de Calidad

Tests unitarios pasando	≥ 10	52 ■
Tests de integración pasando	≥ 1	1 ■
Endpoints funcionando	100%	100% ■
Tiempo de respuesta API	< 2s	< 1s ■
App compila sin errores	Sí	Sí ■
Documentación completa	Sí	Sí ■

3. Procedimiento de Incidencias

Bug	Encoding UTF-8 incorrecto	Debug + fix
Configuración	Puerto 8080 ocupado	Cambiar al 8085
Requisito	Formato matrícula limitado	Actualizar regex

Ciclo de resolución de incidencias:

- 1. Identificación del problema
- 2. Análisis de causa raíz
- 3. Implementación de solución
- 4. Verificación mediante tests
- 5. Documentación de la solución

4. Gestión de Cambios

Se utilizó Git como sistema de control de versiones con commits descriptivos y ramas por funcionalidad (feature/*). Ejemplos de mensajes de commit:

```
fix: encoding UTF-8 en MySQL datasource
feat: escaneo QR con alternativa manual
refactor: ID usuario desde BD en lugar de hashCode
```

5. Resumen de Evaluación

Procedimiento de evaluación	■ Definido
Indicadores de calidad	■ Implementados
Gestión de incidencias	■ Documentado
Control de cambios	■ Con Git
Evaluación de usuarios	■■ Académico (sin cliente real)
Pliego de condiciones	N/A — proyecto académico

PARTE
7**Presentación TFG**

Resumen ejecutivo del proyecto para su presentación ante el tribunal evaluador.

Introducción — El Problema y La Solución

Los talleres mecánicos tradicionales gestionan sus operaciones en papel y libretas, sin conexión entre la oficina y la zona de trabajo, con pérdida de tiempo y dificultad para localizar información.

- Aplicación móvil que digitaliza la gestión al móvil del mecánico
- Digitalización completa: clientes, vehículos, órdenes, citas
- Tecnología QR para identificación rápida de vehículos sin teclear

Objetivos del Proyecto

Objetivo general: desarrollar un CRM completo y útil para dispositivos Android.

- API REST estable con Java y Spring Boot
- Interfaz limpia y fácil de usar en Flutter
- Sistema de autenticación seguro con JWT
- Lector de códigos QR integrado
- Módulo de gestión de citas
- Despliegue con Docker

Tecnologías Utilizadas

Stack tecnológico seleccionado para maximizar la compatibilidad y facilitar el despliegue:

- Frontend: Flutter 3.16+ con Dart y Android Studio
- Backend: Java 17 con Spring Boot 3.2 y Maven
- Base de datos: MySQL 8.0 containerizada con Docker
- Autenticación: JWT con roles ADMINISTRADOR y MECANICO

Resultados Obtenidos

El proyecto cumple con todos los requisitos funcionales establecidos:

- ■ Login con JWT — autenticación segura por roles
- ■ CRUD completo de clientes y vehículos
- ■ Órdenes de reparación con gestión de estados
- ■ Gestión de citas para administradores
- ■ Escáner QR integrado con alternativa manual
- ■ Sistema de ayuda integrado
- ■ 52 tests (unitarios + integración) — todos pasan

Aprendizajes y Desarrollos Futuros

El proyecto ha supuesto una experiencia de desarrollo completo de una app real:

- Integración Flutter ↔ Spring Boot ↔ MySQL ↔ Docker
- Implementación de autenticación JWT
- Resolución de problemas técnicos reales
- Documentación técnica profesional
- Futuro: fotos, notificaciones push, facturación PDF, versión iOS

Valoración Económica

Desarrollo (~300 h a 20-25€/h)	6.000 – 7.500 €
Licencias de software	0 € (todo open-source)
Mantenimiento mensual servidor	30 – 50 €/mes

Contacto

Autora: Nahid Larhziale Oullada
Email: nahidlarhziale@gmail.com
GitHub: <https://github.com/larnahid/gestion-talleres-2000-entrega>

Anexo — Ficheros Incluidos

backend/target/gestion-talleres-backend-1.0.0.jar	Ejecutable del backend
frontend/build/.../app-release.apk	APK de la aplicación Android
docker/init.sql	Script de creación de base de datos
docker/docker-compose.yml	Configuración de contenedores
DOCUMENTACION/	Carpeta con documentación adicional